AISC Practical – Complete Numerical Solutions

Artificial Intelligence and Soft Computing · Semester V · Exam-style solutions to all 14 questions

Contents

- Q1 – BFS (A → G)
- Q2 – DFS (A → G)
- Q3 – Greedy Best-First Search (A → H)
- Q4 – A* Search (A → H)
- Q5 – Perceptron, Delta rule: AND gate
- Q6 – Perceptron, Delta rule: OR gate
- Q7 – Perceptron AND: η = 0.1, 0.3, 0.5 comparison
- Q8 – Kohonen Self-Organizing Map
- Q9 – Fuzzy MFs: washing conditions
- Q10 – Fuzzy MFs: room temperature
- Q11 – Fuzzy MFs: traffic density
- Q12 – Fuzzy MFs: washing conditions (same as Q9)
- Q13 – A* Search (same as Q4)
- Q14 – DFS (same as Q2)

Sources and assumptions used throughout (please read)

ASSUMPTION / NOTE

Methods and terminology follow the reference PDFs: DFS/BFS traversal style, A* table with g(n), h(n), f(n) = g(n) + h(n) and the remark about two equal-cost optimal paths (PDF 3); delta rule $\Delta w = \eta(t - y)x$ with "new weight = old weight + Δw" (PDF 2).

Graph figures of Q1, Q2/Q14 were read from the picture. In the tree of Q2/Q14 the hand-written "O" is read as D and "a" as G.

Q4/Q13: start and goal are not stated; start A and goal H are assumed. Tie-break: smaller h first.

Q5–Q7: the question paper gives no initial weights, bias or learning rate (except η = 0.1, 0.3, 0.5 in Q7). Assumed: w = (0.2, 0.4) (taken from the delta-rule example in PDF 2), b = 0.1, binary step output, η = 0.1 for Q5, η = 0.3 chosen for Q6.

Q8: the data are not given, so a small assumed example is used. **Q9–Q12:** ranges, input values and MF parameters are assumed ("assume suitable input data"). Q9 and Q12 are the same question and are solved completely twice; likewise Q4/Q13 and Q2/Q14.

The reference PDFs contain no numerical values for Q5–Q12, so none were copied from them; every such value is explicitly labelled as assumed.

Q1. Breadth-First Search (BFS): A → G

Question: *Implement BFS to find the path from start node A to goal node G for the given graph (11 edges, drawn as a 2 × 3 grid A–B–C / D–E–F plus node G).*

Step 1: Read the graph from the figure

Edges: A–B, B–C, A–D, B–E, C–F, D–E, E–F, A–E, C–E, C–G, F–G (undirected). Neighbours are taken in **alphabetical order** (assumption for a unique answer).

Node	Neighbours
A	B, D, E
B	A, C, E
C	B, E, F, G
D	A, E
E	A, B, C, D, F
F	C, E, G
G	C, F

Step 2: Method

- BFS uses a **QUEUE (FIFO)**: remove from the front, add new nodes at the rear.
- A node is marked **visited** when it is added to the queue so it is never added twice. Parent of each node is stored to rebuild the path.
- Goal test is done when a node is removed from the queue.

Step 3: Execution (queue shown front → rear)

Step	Remove (current)	Goal?	Neighbours checked	Added to queue (parent)	Queue after step	Visited so far
0	–	–	–	A (start)	[A]	A
1	A	No	B, D, E (all new)	B(A), D(A), E(A)	[B, D, E]	A, B, D, E
2	B	No	A (visited), C (new), E (already in queue)	C(B)	[D, E, C]	A, B, D, E, C
3	D	No	A, E (both already visited)	nothing	[E, C]	A, B, D, E, C
4	E	No	A, B, C, D (visited), F (new)	F(E)	[C, F]	A, B, D, E, C, F
5	C	No	B, E, F (visited), G (new)	G(C)	[F, G]	A, B, D, E, C, F, G
6	F	No	C, E, G (all visited)	nothing	[G]	same
7	G	**YES**	–	–	[ ]	STOP

Step 4: Path reconstruction (follow parents backwards)

G ← C (parent of G) ← B (parent of C) ← A (parent of B) ⇒ reversed: **A → B → C → G**.

FINAL ANSWER

BFS traversal (order of removal from queue): **A → B → D → E → C → F → G**

Path from A to G: **A → B → C → G** (3 edges, the minimum number of edges).

Note: A → E → C → G also has 3 edges; BFS returns the first one it discovers because C was reached through B first.

Q2. Depth-First Search (DFS): A → G

Question: *Implement DFS to find the path from start node A to goal node G for the given tree-shaped graph.*

Step 1: Read the graph from the figure

Tree edges: A–B, A–C, B–D, B–E, E–H, C–F, C–G. (The hand-written labels "O" and "a" in the figure are read as **D** and **G**.) Children are visited **left to right**.

Node	Neighbours
A	B, C
B	D, E
C	F, G
D	– (leaf)
E	H
F	– (leaf)
G	– (leaf, GOAL)
H	– (leaf)

Step 2: Method

- DFS uses a **STACK (LIFO)**: pop the top node, push its children so that the **left child is on top** (push right child first).
- DFS goes as deep as possible; when a leaf is reached it backtracks automatically by popping the next node.

Step 3: Execution (stack shown top → bottom)

Step	Pop (current)	Goal?	Children pushed	Stack after step	Visited order
0	–	–	A pushed	[A]	–
1	A	No	C then B (B on top)	[B, C]	A
2	B	No	E then D (D on top)	[D, E, C]	A, B
3	D	No	none (leaf) → backtrack	[E, C]	A, B, D
4	E	No	H	[H, C]	A, B, D, E
5	H	No	none (leaf) → backtrack	[C]	A, B, D, E, H
6	C	No	G then F (F on top)	[F, G]	A, B, D, E, H, C
7	F	No	none (leaf) → backtrack	[G]	A, B, D, E, H, C, F
8	G	**YES**	–	[ ]	A, B, D, E, H, C, F, G

Step 4: Path from A to G (only the branch that leads to G, ignoring dead ends B, D, E, H, F)

G ← C ← A ⇒ **A → C → G**.

FINAL ANSWER

DFS traversal order: **A → B → D → E → H → C → F → G**

Path found from A to G: **A → C → G** (2 edges). Nodes visited before reaching goal: 8 (including G).

Q3. Greedy Best-First Search: A → H

Question: *Implement Greedy Best-First Search to find a path from A to H. Heuristic values: A(6), B(4), C(3), D(2), E(3), F(2), G(1), H(0). Graph (undirected): A: B, C; B: D, E; C: F, G; D: H; E: H; F: H; G: H.*

Step 1: Data

Node	A	B	C	D	E	F	G	H
h(n)	6	4	3	2	3	2	1	0

Neighbours: A: B, C | B: A, D, E | C: A, F, G | D: B, H | E: B, H | F: C, H | G: C, H | H: D, E, F, G.

Step 2: Method

- Evaluation function **f(n) = h(n)** only (path cost g is ignored).
- **OPEN** list is kept sorted by smallest h(n). Always expand the node with the **smallest h**. Expanded nodes go to **CLOSED**. Nodes already in CLOSED/OPEN are not added again.

Step 3: Execution

Step	OPEN list before selection (node:h)	Select (lowest h)	Goal?	Children added (h)	CLOSED after
1	A:6	A (h=6)	No	B (4), C (3)	A
2	C:3, B:4	C (h=3)	No	F (2), G (1) [A is in CLOSED]	A, C
3	G:1, F:2, B:4	G (h=1)	No	H (0) [C is in CLOSED]	A, C, G
4	H:0, F:2, B:4	H (h=0)	**YES**	–	A, C, G, H

Step 4: Path (follow parents)

H ← G ← C ← A ⇒ **A → C → G → H**. Heuristic sequence along the path: 6 → 3 → 1 → 0 (always decreasing).

FINAL ANSWER

Order of expansion: **A → C → G → H**

Path found: **A → C → G → H** (3 edges). No edge costs are given in this question, so the path length is counted in edges.

Remark: Greedy search is not guaranteed to be optimal. Here A → B → D → H also has 3 edges, so both are equally short.

Q4. A* Search: A → H

Question: *Implement A* Search to find the path from the start node to the goal node. Heuristic: A(5), B(4), C(3), D(3), E(3), F(2), G(1), H(0). Edge costs: A-B 2, A-C 3, B-D 3, B-E 2, C-F 2, C-G 3, D-H 3, E-H 4, F-H 3, G-H 2.*

ASSUMPTION / NOTE

The question does not name the start and goal nodes. From the data (h(H) = 0 and the similar Q3), **start = A and goal = H** is assumed. Tie-break rule (assumed): if two nodes have equal f, select the one with the smaller h (closer to the goal).

Step 1: Data

Node	A	B	C	D	E	F	G	H
h(n)	5	4	3	3	3	2	1	0

Neighbours with edge cost: A: B(2), C(3) | B: A(2), D(3), E(2) | C: A(3), F(2), G(3) | D: B(3), H(3) | E: B(2), H(4) | F: C(2), H(3) | G: C(3), H(2).

Step 2: Formula

f(n) = g(n) + h(n), where g(n) = actual cost from A to n, h(n) = estimated cost from n to H. OPEN is sorted by smallest f. g of a child = g(parent) + edge cost. If a node is reached again with a smaller g it is updated; with an equal or larger g it is ignored.

Step 3: Execution (each OPEN entry is written as node[g + h = f])

Step	OPEN before selection (sorted)	Select	Expansion: children with g, h, f	CLOSED after
1	A[0+5=5]	A	B: g=0+2=2, h=4, **f=6** C: g=0+3=3, h=3, **f=6**	A
2	C[3+3=6], B[2+4=6] (tie f=6, C has smaller h)	C	F: g=3+2=5, h=2, **f=7** G: g=3+3=6, h=1, **f=7** (A already CLOSED)	A, C
3	B[2+4=6], F[5+2=7], G[6+1=7]	B	D: g=2+3=5, h=3, **f=8** E: g=2+2=4, h=3, **f=7** (A already CLOSED)	A, C, B
4	G[6+1=7], F[5+2=7], E[4+3=7], D[5+3=8] (tie f=7: smallest h first → G)	G	H: g=6+2=8, h=0, **f=8** (C already CLOSED)	A, C, B, G
5	F[5+2=7], E[4+3=7], H[8+0=8], D[5+3=8]	F	H again: g=5+3=8, f=8. Equal to existing g=8 ⇒ no improvement, ignored	A, C, B, G, F
6	E[4+3=7], H[8+0=8], D[5+3=8]	E	H again: g=4+4=8, f=8. Equal ⇒ ignored	A, C, B, G, F, E
7	H[8+0=8], D[5+3=8] (tie f=8, H has h=0)	**H = GOAL**	Goal selected for expansion ⇒ STOP	A, C, B, G, F, E, H

Step 4: Path and cost

Parents: H ← G ← C ← A. Path = **A → C → G → H**. Cost g(H) = 3 + 3 + 2 = **8**.

Step 5: Check that 8 is really the minimum (all paths from A to H)

Path	Cost calculation	Total g
A → B → D → H	2 + 3 + 3	8
A → B → E → H	2 + 2 + 4	8
A → C → F → H	3 + 2 + 3	8
A → C → G → H	3 + 3 + 2	8

FINAL ANSWER

Optimal path (as found by A*): **A → C → G → H** with total cost **8**.

Nodes expanded in order: A, C, B, G, F, E, then H selected.

All four paths from A to H cost 8, so each of them is also optimal (like the "two optimal paths" remark in the reference solution); the one returned depends on the tie-break rule.

Q5. Single Layer Perceptron (Delta Rule): AND gate

Question: *Implement a Single Layer Perceptron using the Delta Learning Rule to realize a two-input AND gate. Initialize the weights and bias with small values, apply the training patterns iteratively, and determine the final weights and bias that classify all input combinations correctly.*

Step 1: Truth table of AND

x_1	x_2	Target t (x_1 AND x_2)
0	0	0
0	1	0
1	0	0
1	1	1

Step 2: Rule, initial values and learning rate

Delta learning rule (same form as in reference PDF 2, Q1(c)): $\mathbf{\Delta w_i = \eta\,(t - y)\,x_i}$ and for the bias $\mathbf{\Delta b = \eta\,(t - y)}$; new value = old value + change.

Net input: $\mathbf{Net = b + w_1x_1 + w_2x_2}$. Activation (binary step): $\mathbf{y = 1}$ **if** $\mathbf{Net \geq 0}$, **otherwise** $\mathbf{y = 0}$. Error: e = t − y. **Stopping condition:** stop when one complete epoch (all 4 patterns) has zero error.

ASSUMPTION / NOTE

Initial weights and bias (assumed, "small values"): $\mathbf{w_1 = 0.2}$, $\mathbf{w_2 = 0.4}$, $\mathbf{b = 0.1}$. The pair (0.2, 0.4) is the old weight vector used in the delta-rule example of reference PDF 2 (Q1(c)); the bias 0.1 is my own assumption because no bias is given. Training patterns are applied in the order (0,0), (0,1), (1,0), (1,1) and weights are updated after every pattern.

Learning rate (assumed): $\mathbf{\eta = 0.1}$.

Step 3: Training (every pattern, every epoch)

Epoch 1 (weights at start: w_1=0.2, w_2=0.4, b=0.1)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	0.1 + (0.2)(0) + (0.4)(0)	0.1	1	-1	0.0	0.0	-0.1	0.2	0.4	0.0
2	(0,1)	0	0.0 + (0.2)(0) + (0.4)(1)	0.4	1	-1	0.0	-0.1	-0.1	0.2	0.3	-0.1
3	(1,0)	0	-0.1 + (0.2)(1) + (0.3)(0)	0.1	1	-1	-0.1	0.0	-0.1	0.1	0.3	-0.2
4	(1,1)	1	-0.2 + (0.1)(1) + (0.3)(1)	0.2	1	0	0.0	0.0	0.0	0.1	0.3	-0.2

End of epoch 1: **3** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.1, w_2 = 0.3, b = -0.2.

Epoch 2 (weights at start: w_1=0.1, w_2=0.3, b=-0.2)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.2 + (0.1)(0) + (0.3)(0)	-0.2	0	0	0.0	0.0	0.0	0.1	0.3	-0.2
2	(0,1)	0	-0.2 + (0.1)(0) + (0.3)(1)	0.1	1	-1	0.0	-0.1	-0.1	0.1	0.2	-0.3
3	(1,0)	0	-0.3 + (0.1)(1) + (0.2)(0)	-0.2	0	0	0.0	0.0	0.0	0.1	0.2	-0.3
4	(1,1)	1	-0.3 + (0.1)(1) + (0.2)(1)	0.0	1	0	0.0	0.0	0.0	0.1	0.2	-0.3

End of epoch 2: **1** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.1, w_2 = 0.2, b = -0.3.

Epoch 3 (weights at start: w_1=0.1, w_2=0.2, b=-0.3)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.3 + (0.1)(0) + (0.2)(0)	-0.3	0	0	0.0	0.0	0.0	0.1	0.2	-0.3
2	(0,1)	0	-0.3 + (0.1)(0) + (0.2)(1)	-0.1	0	0	0.0	0.0	0.0	0.1	0.2	-0.3
3	(1,0)	0	-0.3 + (0.1)(1) + (0.2)(0)	-0.2	0	0	0.0	0.0	0.0	0.1	0.2	-0.3
4	(1,1)	1	-0.3 + (0.1)(1) + (0.2)(1)	0.0	1	0	0.0	0.0	0.0	0.1	0.2	-0.3

End of epoch 3: **all 4 patterns correct (no error, no weight change) ⇒ training stops** (converged).

Final verification with the final weights

(x_1,x_2)	Net = -0.3 + 0.1·x_1 + 0.2·x_2	Net value	y (net ≥ 0 → 1)	Target t	Correct?
(0,0)	-0.3 + (0.1)(0) + (0.2)(0)	-0.3	0	0	✓
(0,1)	-0.3 + (0.1)(0) + (0.2)(1)	-0.1	0	0	✓
(1,0)	-0.3 + (0.1)(1) + (0.2)(0)	-0.2	0	0	✓
(1,1)	-0.3 + (0.1)(1) + (0.2)(1)	0.0	1	1	✓

Decision boundary: 0.1 x_1 + 0.2 x_2 + (-0.3) = 0. Only the point (1,1) lies on the "1" side.

FINAL ANSWER

Converged in **3 epochs** (2 epochs with weight changes + 1 final error-free epoch).

Final weights: **w_1 = 0.1, w_2 = 0.2**, bias **b = -0.3** (η = 0.1).

All four AND patterns are classified correctly (see verification table).

Q6. Single Layer Perceptron (Delta Rule): OR gate

Question: *Develop a Single Layer Perceptron using the Delta Learning Rule to implement a two-input OR gate. Train the network using all binary input combinations, experiment with a suitable learning rate, and evaluate convergence by comparing predicted outputs with target outputs.*

Step 1: Truth table of OR

x_1	x_2	Target t (x_1 OR x_2)
0	0	0
0	1	1
1	0	1
1	1	1

Step 2: Rule, initial values

Delta learning rule (same form as in reference PDF 2, Q1(c)): $\mathbf{\Delta w_i = \eta\,(t - y)\,x_i}$ and for the bias $\mathbf{\Delta b = \eta\,(t - y)}$; new value = old value + change.

Net input: $\mathbf{Net = b + w_1x_1 + w_2x_2}$. Activation (binary step): $\mathbf{y = 1}$ **if** $\mathbf{Net \geq 0}$, **otherwise** $\mathbf{y = 0}$. Error: e = t − y. **Stopping condition:** stop when one complete epoch (all 4 patterns) has zero error.

ASSUMPTION / NOTE

Initial weights and bias (assumed, "small values"): $\mathbf{w_1 = 0.2}$, $\mathbf{w_2 = 0.4}$, $\mathbf{b = 0.1}$. The pair (0.2, 0.4) is the old weight vector used in the delta-rule example of reference PDF 2 (Q1(c)); the bias 0.1 is my own assumption because no bias is given. Training patterns are applied in the order (0,0), (0,1), (1,0), (1,1) and weights are updated after every pattern.

Step 3: Experiment with learning rates (same initial values)

η	Epochs needed (incl. final error-free epoch)	Epochs with updates	Final w_1	Final w_2	Final b
0.1	3	2	0.2	0.4	-0.1
0.3	2	1	0.2	0.4	-0.2
0.5	3	2	0.7	0.4	-0.4

All three learning rates converge. $\mathbf{\eta = 0.3}$ is selected as the suitable rate because it needs the fewest epochs (2). The complete training with η = 0.3 is shown below. Epoch-wise trajectory of the other two rates is given at the end.

Step 4: Complete training with η = 0.3

Epoch 1 (weights at start: w_1=0.2, w_2=0.4, b=0.1)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	0.1 + (0.2)(0) + (0.4)(0)	0.1	1	-1	0.0	0.0	-0.3	0.2	0.4	-0.2
2	(0,1)	1	-0.2 + (0.2)(0) + (0.4)(1)	0.2	1	0	0.0	0.0	0.0	0.2	0.4	-0.2
3	(1,0)	1	-0.2 + (0.2)(1) + (0.4)(0)	0.0	1	0	0.0	0.0	0.0	0.2	0.4	-0.2
4	(1,1)	1	-0.2 + (0.2)(1) + (0.4)(1)	0.4	1	0	0.0	0.0	0.0	0.2	0.4	-0.2

End of epoch 1: **1** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.2, w_2 = 0.4, b = -0.2.

Epoch 2 (weights at start: w_1=0.2, w_2=0.4, b=-0.2)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.2 + (0.2)(0) + (0.4)(0)	-0.2	0	0	0.0	0.0	0.0	0.2	0.4	-0.2
2	(0,1)	1	-0.2 + (0.2)(0) + (0.4)(1)	0.2	1	0	0.0	0.0	0.0	0.2	0.4	-0.2
3	(1,0)	1	-0.2 + (0.2)(1) + (0.4)(0)	0.0	1	0	0.0	0.0	0.0	0.2	0.4	-0.2
4	(1,1)	1	-0.2 + (0.2)(1) + (0.4)(1)	0.4	1	0	0.0	0.0	0.0	0.2	0.4	-0.2

End of epoch 2: **all 4 patterns correct (no error, no weight change)** ⇒ **training stops** (converged).

Final verification with the final weights

(x_1,x_2)	Net = -0.2 + 0.2·x_1 + 0.4·x_2	Net value	y (net ≥ 0 → 1)	Target t	Correct?
(0,0)	-0.2 + (0.2)(0) + (0.4)(0)	-0.2	0	0	✓
(0,1)	-0.2 + (0.2)(0) + (0.4)(1)	0.2	1	1	✓
(1,0)	-0.2 + (0.2)(1) + (0.4)(0)	0.0	1	1	✓
(1,1)	-0.2 + (0.2)(1) + (0.4)(1)	0.4	1	1	✓

Step 5: Convergence evaluation (predicted output vs target output)

(x_1,x_2)	Predicted y	Target t	Error t−y
(0,0)	0	0	0
(0,1)	1	1	0
(1,0)	1	1	0
(1,1)	1	1	0

Total error = 0 in the last epoch ⇒ the network has converged.

Weights at the end of each epoch for the other learning rates

η	Epoch	Errors in epoch	w_1	w_2	b
0.1	1	1	0.2	0.4	0.0
0.1	2	1	0.2	0.4	-0.1
0.1	3	0	0.2	0.4	-0.1
0.5	1	2	0.7	0.4	0.1
0.5	2	1	0.7	0.4	-0.4
0.5	3	0	0.7	0.4	-0.4

FINAL ANSWER

Best learning rate in this experiment: **η = 0.3**, converged in **2 epochs** (1 epoch with changes + 1 error-free epoch).

Final weights: **w_1 = 0.2, w_2 = 0.4, b = -0.2**.

Predicted outputs 0, 1, 1, 1 equal targets 0, 1, 1, 1 for all combinations.

Q7. Perceptron (Delta Rule): AND gate with η = 0.1, 0.3, 0.5

Question: *Implement a Single Layer Perceptron for the AND gate using the Delta Learning Rule. Train separately with learning rates 0.1, 0.3 and 0.5, keeping initial weights and training patterns identical. Compare the number of epochs required for convergence and analyze the effect of learning rate.*

Step 1: Truth table of AND

x_1	x_2	Target t (x_1 AND x_2)
0	0	0
0	1	0
1	0	0
1	1	1

Step 2: Rule and common initial values

Delta learning rule (same form as in reference PDF 2, Q1(c)): $\mathbf{\Delta w_i = \eta\,(t - y)\,x_i}$ and for the bias $\mathbf{\Delta b = \eta\,(t - y)}$; new value = old value + change.

Net input: $\mathbf{Net = b + w_1x_1 + w_2x_2}$. Activation (binary step): $\mathbf{y = 1}$ **if** $\mathbf{Net \geq 0}$, **otherwise** $\mathbf{y = 0}$. Error: e = t − y. **Stopping condition:** stop when one complete epoch (all 4 patterns) has zero error.

ASSUMPTION / NOTE

Initial weights and bias (assumed, "small values"): $\mathbf{w_1 = 0.2}$, $\mathbf{w_2 = 0.4}$, $\mathbf{b = 0.1}$. The pair (0.2, 0.4) is the old weight vector used in the delta-rule example of reference PDF 2 (Q1(c)); the bias 0.1 is my own assumption because no bias is given. Training patterns are applied in the order (0,0), (0,1), (1,0), (1,1) and weights are updated after every pattern.

Step 3: Training with η = 0.1

Epoch 1 (weights at start: w_1=0.2, w_2=0.4, b=0.1)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	0.1 + (0.2)(0) + (0.4)(0)	0.1	1	-1	0.0	0.0	-0.1	0.2	0.4	0.0
2	(0,1)	0	0.0 + (0.2)(0) + (0.4)(1)	0.4	1	-1	0.0	-0.1	-0.1	0.2	0.3	-0.1
3	(1,0)	0	-0.1 + (0.2)(1) + (0.3)(0)	0.1	1	-1	-0.1	0.0	-0.1	0.1	0.3	-0.2
4	(1,1)	1	-0.2 + (0.1)(1) + (0.3)(1)	0.2	1	0	0.0	0.0	0.0	0.1	0.3	-0.2

End of epoch 1: **3** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.1, w_2 = 0.3, b = -0.2.

Epoch 2 (weights at start: w_1=0.1, w_2=0.3, b=-0.2)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.2 + (0.1)(0) + (0.3)(0)	-0.2	0	0	0.0	0.0	0.0	0.1	0.3	-0.2
2	(0,1)	0	-0.2 + (0.1)(0) + (0.3)(1)	0.1	1	-1	0.0	-0.1	-0.1	0.1	0.2	-0.3
3	(1,0)	0	-0.3 + (0.1)(1) + (0.2)(0)	-0.2	0	0	0.0	0.0	0.0	0.1	0.2	-0.3
4	(1,1)	1	-0.3 + (0.1)(1) + (0.2)(1)	0.0	1	0	0.0	0.0	0.0	0.1	0.2	-0.3

End of epoch 2: **1** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.1, w_2 = 0.2, b = -0.3.

Epoch 3 (weights at start: w_1=0.1, w_2=0.2, b=-0.3)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.3 + (0.1)(0) + (0.2)(0)	-0.3	0	0	0.0	0.0	0.0	0.1	0.2	-0.3
2	(0,1)	0	-0.3 + (0.1)(0) + (0.2)(1)	-0.1	0	0	0.0	0.0	0.0	0.1	0.2	-0.3
3	(1,0)	0	-0.3 + (0.1)(1) + (0.2)(0)	-0.2	0	0	0.0	0.0	0.0	0.1	0.2	-0.3
4	(1,1)	1	-0.3 + (0.1)(1) + (0.2)(1)	0.0	1	0	0.0	0.0	0.0	0.1	0.2	-0.3

End of epoch 3: **all 4 patterns correct (no error, no weight change)** ⇒ **training stops** (converged).

Final verification with the final weights

(x_1,x_2)	Net = -0.3 + 0.1·x_1 + 0.2·x_2	Net value	y (net ≥ 0 → 1)	Target t	Correct?
(0,0)	-0.3 + (0.1)(0) + (0.2)(0)	-0.3	0	0	✓
(0,1)	-0.3 + (0.1)(0) + (0.2)(1)	-0.1	0	0	✓
(1,0)	-0.3 + (0.1)(1) + (0.2)(0)	-0.2	0	0	✓
(1,1)	-0.3 + (0.1)(1) + (0.2)(1)	0.0	1	1	✓

Result for η = 0.1: converged in **3 epochs**; final w_1 = 0.1, w_2 = 0.2, b = -0.3.

Step 4: Training with η = 0.3

Epoch 1 (weights at start: w_1=0.2, w_2=0.4, b=0.1)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	0.1 + (0.2)(0) + (0.4)(0)	0.1	1	-1	0.0	0.0	-0.3	0.2	0.4	-0.2
2	(0,1)	0	-0.2 + (0.2)(0) + (0.4)(1)	0.2	1	-1	0.0	-0.3	-0.3	0.2	0.1	-0.5
3	(1,0)	0	-0.5 + (0.2)(1) + (0.1)(0)	-0.3	0	0	0.0	0.0	0.0	0.2	0.1	-0.5
4	(1,1)	1	-0.5 + (0.2)(1) + (0.1)(1)	-0.2	0	+1	0.3	0.3	0.3	0.5	0.4	-0.2

End of epoch 1: **3** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.5, w_2 = 0.4, b = -0.2.

Epoch 2 (weights at start: w_1=0.5, w_2=0.4, b=-0.2)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.2 + (0.5)(0) + (0.4)(0)	-0.2	0	0	0.0	0.0	0.0	0.5	0.4	-0.2
2	(0,1)	0	-0.2 + (0.5)(0) + (0.4)(1)	0.2	1	-1	0.0	-0.3	-0.3	0.5	0.1	-0.5
3	(1,0)	0	-0.5 + (0.5)(1) + (0.1)(0)	0.0	1	-1	-0.3	0.0	-0.3	0.2	0.1	-0.8
4	(1,1)	1	-0.8 + (0.2)(1) + (0.1)(1)	-0.5	0	+1	0.3	0.3	0.3	0.5	0.4	-0.5

End of epoch 2: **3** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.5, w_2 = 0.4, b = -0.5.

Epoch 3 (weights at start: w_1=0.5, w_2=0.4, b=-0.5)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.5 + (0.5)(0) + (0.4)(0)	-0.5	0	0	0.0	0.0	0.0	0.5	0.4	-0.5
2	(0,1)	0	-0.5 + (0.5)(0) + (0.4)(1)	-0.1	0	0	0.0	0.0	0.0	0.5	0.4	-0.5
3	(1,0)	0	-0.5 + (0.5)(1) + (0.4)(0)	0.0	1	-1	-0.3	0.0	-0.3	0.2	0.4	-0.8
4	(1,1)	1	-0.8 + (0.2)(1) + (0.4)(1)	-0.2	0	+1	0.3	0.3	0.3	0.5	0.7	-0.5

End of epoch 3: **2** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.5, w_2 = 0.7, b = -0.5.

Epoch 4 (weights at start: w_1=0.5, w_2=0.7, b=-0.5)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.5 + (0.5)(0) + (0.7)(0)	-0.5	0	0	0.0	0.0	0.0	0.5	0.7	-0.5
2	(0,1)	0	-0.5 + (0.5)(0) + (0.7)(1)	0.2	1	-1	0.0	-0.3	-0.3	0.5	0.4	-0.8
3	(1,0)	0	-0.8 + (0.5)(1) + (0.4)(0)	-0.3	0	0	0.0	0.0	0.0	0.5	0.4	-0.8
4	(1,1)	1	-0.8 + (0.5)(1) + (0.4)(1)	0.1	1	0	0.0	0.0	0.0	0.5	0.4	-0.8

End of epoch 4: **1** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.5, w_2 = 0.4, b = -0.8.

Epoch 5 (weights at start: w_1=0.5, w_2=0.4, b=-0.8)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.8 + (0.5)(0) + (0.4)(0)	-0.8	0	0	0.0	0.0	0.0	0.5	0.4	-0.8
2	(0,1)	0	-0.8 + (0.5)(0) + (0.4)(1)	-0.4	0	0	0.0	0.0	0.0	0.5	0.4	-0.8
3	(1,0)	0	-0.8 + (0.5)(1) + (0.4)(0)	-0.3	0	0	0.0	0.0	0.0	0.5	0.4	-0.8
4	(1,1)	1	-0.8 + (0.5)(1) + (0.4)(1)	0.1	1	0	0.0	0.0	0.0	0.5	0.4	-0.8

End of epoch 5: **all 4 patterns correct (no error, no weight change)** ⇒ **training stops** (converged).

Final verification with the final weights

(x_1,x_2)	Net = -0.8 + 0.5·x_1 + 0.4·x_2	Net value	y (net ≥ 0 → 1)	Target t	Correct?
(0,0)	-0.8 + (0.5)(0) + (0.4)(0)	-0.8	0	0	✓
(0,1)	-0.8 + (0.5)(0) + (0.4)(1)	-0.4	0	0	✓
(1,0)	-0.8 + (0.5)(1) + (0.4)(0)	-0.3	0	0	✓
(1,1)	-0.8 + (0.5)(1) + (0.4)(1)	0.1	1	1	✓

Result for η = 0.3: converged in **5 epochs**; final w_1 = 0.5, w_2 = 0.4, b = -0.8.

Step 5: Training with η = 0.5

Epoch 1 (weights at start: w_1=0.2, w_2=0.4, b=0.1)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	0.1 + (0.2)(0) + (0.4)(0)	0.1	1	-1	0.0	0.0	-0.5	0.2	0.4	-0.4
2	(0,1)	0	-0.4 + (0.2)(0) + (0.4)(1)	0.0	1	-1	0.0	-0.5	-0.5	0.2	-0.1	-0.9
3	(1,0)	0	-0.9 + (0.2)(1) + (-0.1)(0)	-0.7	0	0	0.0	0.0	0.0	0.2	-0.1	-0.9
4	(1,1)	1	-0.9 + (0.2)(1) + (-0.1)(1)	-0.8	0	+1	0.5	0.5	0.5	0.7	0.4	-0.4

End of epoch 1: **3** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 0.7, w_2 = 0.4, b = -0.4.

Epoch 2 (weights at start: w_1=0.7, w_2=0.4, b=-0.4)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.4 + (0.7)(0) + (0.4)(0)	-0.4	0	0	0.0	0.0	0.0	0.7	0.4	-0.4
2	(0,1)	0	-0.4 + (0.7)(0) + (0.4)(1)	0.0	1	-1	0.0	-0.5	-0.5	0.7	-0.1	-0.9
3	(1,0)	0	-0.9 + (0.7)(1) + (-0.1)(0)	-0.2	0	0	0.0	0.0	0.0	0.7	-0.1	-0.9
4	(1,1)	1	-0.9 + (0.7)(1) + (-0.1)(1)	-0.3	0	+1	0.5	0.5	0.5	1.2	0.4	-0.4

End of epoch 2: **2** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 1.2, w_2 = 0.4, b = -0.4.

Epoch 3 (weights at start: w_1=1.2, w_2=0.4, b=-0.4)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.4 + (1.2)(0) + (0.4)(0)	-0.4	0	0	0.0	0.0	0.0	1.2	0.4	-0.4
2	(0,1)	0	-0.4 + (1.2)(0) + (0.4)(1)	0.0	1	-1	0.0	-0.5	-0.5	1.2	-0.1	-0.9
3	(1,0)	0	-0.9 + (1.2)(1) + (-0.1)(0)	0.3	1	-1	-0.5	0.0	-0.5	0.7	-0.1	-1.4
4	(1,1)	1	-1.4 + (0.7)(1) + (-0.1)(1)	-0.8	0	+1	0.5	0.5	0.5	1.2	0.4	-0.9

End of epoch 3: **3** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 1.2, w_2 = 0.4, b = -0.9.

Epoch 4 (weights at start: w_1=1.2, w_2=0.4, b=-0.9)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.9 + (1.2)(0) + (0.4)(0)	-0.9	0	0	0.0	0.0	0.0	1.2	0.4	-0.9
2	(0,1)	0	-0.9 + (1.2)(0) + (0.4)(1)	-0.5	0	0	0.0	0.0	0.0	1.2	0.4	-0.9
3	(1,0)	0	-0.9 + (1.2)(1) + (0.4)(0)	0.3	1	-1	-0.5	0.0	-0.5	0.7	0.4	-1.4
4	(1,1)	1	-1.4 + (0.7)(1) + (0.4)(1)	-0.3	0	+1	0.5	0.5	0.5	1.2	0.9	-0.9

End of epoch 4: **2** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 1.2, w_2 = 0.9, b = -0.9.

Epoch 5 (weights at start: w_1=1.2, w_2=0.9, b=-0.9)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-0.9 + (1.2)(0) + (0.9)(0)	-0.9	0	0	0.0	0.0	0.0	1.2	0.9	-0.9
2	(0,1)	0	-0.9 + (1.2)(0) + (0.9)(1)	0.0	1	-1	0.0	-0.5	-0.5	1.2	0.4	-1.4
3	(1,0)	0	-1.4 + (1.2)(1) + (0.4)(0)	-0.2	0	0	0.0	0.0	0.0	1.2	0.4	-1.4
4	(1,1)	1	-1.4 + (1.2)(1) + (0.4)(1)	0.2	1	0	0.0	0.0	0.0	1.2	0.4	-1.4

End of epoch 5: **1** pattern(s) gave an error, so weights were changed. Weights carried to next epoch: w_1 = 1.2, w_2 = 0.4, b = -1.4.

Epoch 6 (weights at start: w_1=1.2, w_2=0.4, b=-1.4)

#	(x_1, x_2)	t	Net = b + w_1x_1 + w_2x_2 (substitution)	Net	y	e = t−y	Δw_1	Δw_2	Δb	w_1 new	w_2 new	b new
1	(0,0)	0	-1.4 + (1.2)(0) + (0.4)(0)	-1.4	0	0	0.0	0.0	0.0	1.2	0.4	-1.4
2	(0,1)	0	-1.4 + (1.2)(0) + (0.4)(1)	-1.0	0	0	0.0	0.0	0.0	1.2	0.4	-1.4
3	(1,0)	0	-1.4 + (1.2)(1) + (0.4)(0)	-0.2	0	0	0.0	0.0	0.0	1.2	0.4	-1.4
4	(1,1)	1	-1.4 + (1.2)(1) + (0.4)(1)	0.2	1	0	0.0	0.0	0.0	1.2	0.4	-1.4

End of epoch 6: **all 4 patterns correct (no error, no weight change) ⇒ training stops** (converged).

Final verification with the final weights

(x_1,x_2)	Net = -1.4 + 1.2·x_1 + 0.4·x_2	Net value	y (net ≥ 0 → 1)	Target t	Correct?
(0,0)	-1.4 + (1.2)(0) + (0.4)(0)	-1.4	0	0	✓
(0,1)	-1.4 + (1.2)(0) + (0.4)(1)	-1.0	0	0	✓
(1,0)	-1.4 + (1.2)(1) + (0.4)(0)	-0.2	0	0	✓
(1,1)	-1.4 + (1.2)(1) + (0.4)(1)	0.2	1	1	✓

Result for η = 0.5: converged in **6 epochs**; final w_1 = 1.2, w_2 = 0.4, b = -1.4.

Step 6: Comparison table

Learning rate η	Epochs with weight updates	Total epochs to converge (incl. final error-free epoch)	Total weight updates	Final w_1	Final w_2	Final b
0.1	2	**3**	4	0.1	0.2	-0.3
0.3	4	**5**	9	0.5	0.4	-0.8
0.5	5	**6**	11	1.2	0.4	-1.4

Step 7: Analysis of the effect of the learning rate

- The learning rate η multiplies every correction (Δw = η(t−y)x). A **small η** gives small, careful corrections; a **large η** gives large corrections.
- In this experiment (fixed initial values) the number of epochs **increased from 3 (η = 0.1) to 5 (η = 0.3) and 6 (η = 0.5)**. With larger η the weights jump further in each update, overshoot the correct region and need extra corrections (the bias moved to -0.8 and -1.4, far beyond the -0.3 needed for η = 0.1).
- All three rates converge because AND is linearly separable (perceptron convergence theorem). The final weights differ, but all of them classify the four patterns correctly.
- This trend belongs to this particular initial choice; with a hard-step perceptron the epoch count depends on the starting weights as well, so it is not a universal law. Very small η can be slow, very large η can overshoot.

FINAL ANSWER

Epochs to converge: **η = 0.1 → 3 epochs, η = 0.3 → 5 epochs, η = 0.5 → 6 epochs** (initial w = (0.2, 0.4), b = 0.1).

Smaller learning rate converged fastest here; larger rates made bigger jumps and needed more epochs. All classify AND correctly.

Q8. Kohonen Self-Organizing Map (SOM)

Question: *Implement the Kohonen SOM algorithm to cluster a given set of input patterns into groups and visualize the clusters. Assume suitable input data.*

Step 1: Assumed example (data are not given in the question)

ASSUMPTION / NOTE

Assumed input data (4 patterns, 2 features) and a map of **2 output neurons** (2 clusters). Initial weights, learning rate and number of epochs are also assumed.

Pattern	x_1	x_2
X1	0.1	0.2
X2	0.2	0.1
X3	0.8	0.9
X4	0.9	0.8

Item	Assumed value
Initial weight of neuron 1	W1 = (0.3, 0.3)
Initial weight of neuron 2	W2 = (0.6, 0.6)
Learning rate α	α(0) = 0.5, halved after every epoch (0.5, 0.25, 0.125)
Neighbourhood radius	R = 0 (only the winning neuron is updated)
Epochs	3

Step 2: Algorithm

- For each input X: compute the **squared Euclidean distance** to every neuron j: $D(j) = (w_{1j} - x_1)^2 + (w_{2j} - x_2)^2$ (Euclidean distance d = √D; comparing D is enough).
- The neuron with the **smallest distance** is the **winner**.
- Update only the winner: $\mathbf{w_{new} = w_{old} + \alpha\,(x - w_{old})}$.
- After each epoch reduce α. Repeat until weights are stable.

Step 3: Calculations (all numbers rounded to 4 decimals)

Epoch 1 (α = 0.5)

Input	D(1) = distance² to neuron 1	D(2) = distance² to neuron 2	Winner	Weight update of winner	New W1 | W2
X1 (0.1, 0.2)	$(0.3-0.1)^2 + (0.3-0.2)^2$ = 0.0400 + 0.0100 = **0.0500** d = √0.0500 = 0.2236	$(0.6-0.1)^2 + (0.6-0.2)^2$ = 0.2500 + 0.1600 = **0.4100** d = √0.4100 = 0.6403	**Neuron 1** (0.0500 < 0.4100)	w_1: 0.3 + 0.5(0.1 − 0.3) = 0.2000 w_2: 0.3 + 0.5(0.2 − 0.3) = 0.2500	(0.2000, 0.2500) | (0.6000, 0.6000)
X2 (0.2, 0.1)	$(0.2-0.2)^2 + (0.25-0.1)^2$ = 0.0000 + 0.0225 = **0.0225** d = √0.0225 = 0.1500	$(0.6-0.2)^2 + (0.6-0.1)^2$ = 0.1600 + 0.2500 = **0.4100** d = √0.4100 = 0.6403	**Neuron 1** (0.0225 < 0.4100)	w_1: 0.2 + 0.5(0.2 − 0.2) = 0.2000 w_2: 0.25 + 0.5(0.1 − 0.25) = 0.1750	(0.2000, 0.1750) | (0.6000, 0.6000)
X3 (0.8, 0.9)	$(0.2-0.8)^2 + (0.175-0.9)^2$ = 0.3600 + 0.5256 = **0.8856** d = √0.8856 = 0.9411	$(0.6-0.8)^2 + (0.6-0.9)^2$ = 0.0400 + 0.0900 = **0.1300** d = √0.1300 = 0.3606	**Neuron 2** (0.1300 < 0.8856)	w_1: 0.6 + 0.5(0.8 − 0.6) = 0.7000 w_2: 0.6 + 0.5(0.9 − 0.6) = 0.7500	(0.2000, 0.1750) | (0.7000, 0.7500)
X4 (0.9, 0.8)	$(0.2-0.9)^2 + (0.175-0.8)^2$ = 0.4900 + 0.3906 = **0.8806** d = √0.8806 = 0.9384	$(0.7-0.9)^2 + (0.75-0.8)^2$ = 0.0400 + 0.0025 = **0.0425** d = √0.0425 = 0.2062	**Neuron 2** (0.0425 < 0.8806)	w_1: 0.7 + 0.5(0.9 − 0.7) = 0.8000 w_2: 0.75 + 0.5(0.8 − 0.75) = 0.7750	(0.2000, 0.1750) | (0.8000, 0.7750)

Weights at end of epoch 1: W1 = (0.2000, 0.1750), W2 = (0.8000, 0.7750). Next α = α/2 = 0.25.

Epoch 2 (α = 0.25)

Input	D(1) = distance² to neuron 1	D(2) = distance² to neuron 2	Winner	Weight update of winner	New W1 | W2
X1 (0.1, 0.2)	$(0.2-0.1)^2 + (0.175-0.2)^2$ = 0.0100 + 0.0006 = **0.0106** d = √0.0106 = 0.1030	$(0.8-0.1)^2 + (0.775-0.2)^2$ = 0.4900 + 0.3306 = **0.8206** d = √0.8206 = 0.9059	**Neuron 1** (0.0106 < 0.8206)	w_1: 0.2 + 0.25(0.1 − 0.2) = 0.1750 w_2: 0.175 + 0.25(0.2 − 0.175) = 0.1813	(0.1750, 0.1813) | (0.8000, 0.7750)

Input	D(1) = distance² to neuron 1	D(2) = distance² to neuron 2	Winner	Weight update of winner	New W1 | W2
X2 (0.2, 0.1)	(0.175−0.2)² + (0.1813−0.1)² = 0.0006 + 0.0066 = **0.0072** d = √0.0072 = 0.0849	(0.8−0.2)² + (0.775−0.1)² = 0.3600 + 0.4556 = **0.8156** d = √0.8156 = 0.9031	**Neuron 1** (0.0072 < 0.8156)	w_1: 0.175 + 0.25(0.2 − 0.175) = 0.1813 w_2: 0.1813 + 0.25(0.1 − 0.1813) = 0.1610	(0.1813, 0.1610) | (0.8000, 0.7750)
X3 (0.8, 0.9)	(0.1813−0.8)² + (0.161−0.9)² = 0.3828 + 0.5461 = **0.9289** d = √0.9289 = 0.9638	(0.8−0.8)² + (0.775−0.9)² = 0.0000 + 0.0156 = **0.0156** d = √0.0156 = 0.1249	**Neuron 2** (0.0156 < 0.9289)	w_1: 0.8 + 0.25(0.8 − 0.8) = 0.8000 w_2: 0.775 + 0.25(0.9 − 0.775) = 0.8063	(0.1813, 0.1610) | (0.8000, 0.8063)
X4 (0.9, 0.8)	(0.1813−0.9)² + (0.161−0.8)² = 0.5165 + 0.4083 = **0.9248** d = √0.9248 = 0.9617	(0.8−0.9)² + (0.8063−0.8)² = 0.0100 + 0.0000 = **0.0100** d = √0.0100 = 0.1000	**Neuron 2** (0.0100 < 0.9248)	w_1: 0.8 + 0.25(0.9 − 0.8) = 0.8250 w_2: 0.8063 + 0.25(0.8 − 0.8063) = 0.8047	(0.1813, 0.1610) | (0.8250, 0.8047)

Weights at end of epoch 2: W1 = (0.1813, 0.1610), W2 = (0.8250, 0.8047). Next α = α/2 = 0.125.

Epoch 3 (α = 0.125)

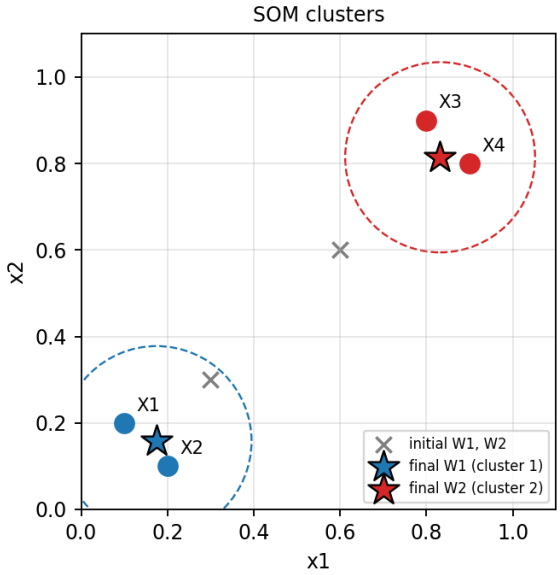
Input	D(1) = distance² to neuron 1	D(2) = distance² to neuron 2	Winner	Weight update of winner	New W1 | W2
X1 (0.1, 0.2)	(0.1813−0.1)² + (0.161−0.2)² = 0.0066 + 0.0015 = **0.0081** d = √0.0081 = 0.0900	(0.825−0.1)² + (0.8047−0.2)² = 0.5256 + 0.3657 = **0.8913** d = √0.8913 = 0.9441	**Neuron 1** (0.0081 < 0.8913)	w_1: 0.1813 + 0.125(0.1 − 0.1813) = 0.1711 w_2: 0.161 + 0.125(0.2 − 0.161) = 0.1659	(0.1711, 0.1659) | (0.8250, 0.8047)
X2 (0.2, 0.1)	(0.1711−0.2)² + (0.1659−0.1)² = 0.0008 + 0.0043 = **0.0051** d = √0.0051 = 0.0714	(0.825−0.2)² + (0.8047−0.1)² = 0.3906 + 0.4966 = **0.8872** d = √0.8872 = 0.9419	**Neuron 1** (0.0051 < 0.8872)	w_1: 0.1711 + 0.125(0.2 − 0.1711) = 0.1747 w_2: 0.1659 + 0.125(0.1 − 0.1659) = 0.1577	(0.1747, 0.1577) | (0.8250, 0.8047)
X3 (0.8, 0.9)	(0.1747−0.8)² + (0.1577−0.9)² = 0.3910 + 0.5510 = **0.9420** d = √0.9420 = 0.9706	(0.825−0.8)² + (0.8047−0.9)² = 0.0006 + 0.0091 = **0.0097** d = √0.0097 = 0.0985	**Neuron 2** (0.0097 < 0.9420)	w_1: 0.825 + 0.125(0.8 − 0.825) = 0.8219 w_2: 0.8047 + 0.125(0.9 − 0.8047) = 0.8166	(0.1747, 0.1577) | (0.8219, 0.8166)
X4 (0.9, 0.8)	(0.1747−0.9)² + (0.1577−0.8)² = 0.5261 + 0.4125 = **0.9386** d = √0.9386 = 0.9688	(0.8219−0.9)² + (0.8166−0.8)² = 0.0061 + 0.0003 = **0.0064** d = √0.0064 = 0.0800	**Neuron 2** (0.0064 < 0.9386)	w_1: 0.8219 + 0.125(0.9 − 0.8219) = 0.8317 w_2: 0.8166 + 0.125(0.8 − 0.8166) = 0.8145	(0.1747, 0.1577) | (0.8317, 0.8145)

Weights at end of epoch 3: W1 = (0.1747, 0.1577), W2 = (0.8317, 0.8145). Next α = α/2 = 0.0625.

Step 4: Final clustering with the final weights

Input	D(1) to W1 = (0.1747, 0.1577)	D(2) to W2 = (0.8317, 0.8145)	Winner = cluster
X1 (0.1, 0.2)	0.0074	0.9130	**Cluster 1** (neuron 1)
X2 (0.2, 0.1)	0.0040	0.9096	**Cluster 1** (neuron 1)
X3 (0.8, 0.9)	0.9420	0.0083	**Cluster 2** (neuron 2)
X4 (0.9, 0.8)	0.9386	0.0049	**Cluster 2** (neuron 2)

Step 5: Visualization of the clusters



FINAL ANSWER

Cluster 1 (neuron 1): **{X1, X2}** — Cluster 2 (neuron 2): **{X3, X4}**

Final weights: W1 = (0.1747, 0.1577), W2 = (0.8317, 0.8145) (each neuron has moved to the centre of its group).

The SOM grouped the low-value patterns together and the high-value patterns together (assumed example).

Q9. Fuzzy Membership Functions: Washing Machine (Soil Level)

Question: *Implement Triangular, Trapezoidal and Gaussian Fuzzy Membership Functions to represent washing conditions (Lightly Soiled, Moderately Soiled, Heavily Soiled) in a fuzzy logic-based washing machine system. Assume suitable input data and plot the corresponding membership functions.*

Step 1: Choose the universe of discourse and the fuzzy sets

Input variable: **Degree of dirt (soil level)**, measured in **% dirt**. Universe of discourse: $x \in [0, 100]$. Fuzzy sets (linguistic terms): **Lightly Soiled, Moderately Soiled, Heavily Soiled**. Each term is represented by three types of membership function (MF).

Step 2: Standard formulas used

MF type	Parameters	Formula
Triangular	a, b, c ($a \le b \le c$)	$\mu(x) = 0$ if $x \le a$ or $x \ge c$; $(x-a)/(b-a)$ if $a \le x \le b$; $(c-x)/(c-b)$ if $b \le x \le c$
Trapezoidal	a, b, c, d ($a \le b \le c \le d$)	$\mu(x) = 0$ if $x \le a$ or $x \ge d$; $(x-a)/(b-a)$ if $a \le x \le b$; 1 if $b \le x \le c$; $(d-x)/(d-c)$ if $c \le x \le d$
Gaussian	c (centre), σ (spread)	$\mu(x) = \exp[\ -(x - c)^2 / (2\sigma^2)\ ]$

Step 3: Assumed parameters and the resulting formula of every membership function

ASSUMPTION / NOTE

The question says "assume suitable input data", so the ranges and parameters below are **assumed** (they are not given in the question paper or reference PDFs). They are chosen so that neighbouring sets overlap, which is the basic idea of fuzzy logic.

MF type	Fuzzy set	Parameters	Formula with numbers substituted
Triangular	Lightly Soiled	(a, b, c) = (0, 0, 50)	$(50 - x)/50$ for $0 \le x \le 50$ 0 elsewhere
Triangular	Moderately Soiled	(a, b, c) = (25, 50, 75)	$(x - 25)/25$ for $25 \le x \le 50$ $(75 - x)/25$ for $50 \le x \le 75$ 0 elsewhere
Triangular	Heavily Soiled	(a, b, c) = (50, 100, 100)	$(x - 50)/50$ for $50 \le x \le 100$ 0 elsewhere
Trapezoidal	Lightly Soiled	(a, b, c, d) = (0, 0, 20, 50)	1 for $0 \le x \le 20$ $(50 - x)/30$ for $20 \le x \le 50$ 0 elsewhere
Trapezoidal	Moderately Soiled	(a, b, c, d) = (25, 40, 60, 75)	$(x - 25)/15$ for $25 \le x \le 40$ 1 for $40 \le x \le 60$ $(75 - x)/15$ for $60 \le x \le 75$ 0 elsewhere
Trapezoidal	Heavily Soiled	(a, b, c, d) = (50, 80, 100, 100)	$(x - 50)/30$ for $50 \le x \le 80$ 1 for $80 \le x \le 100$ 0 elsewhere
Gaussian	Lightly Soiled	(c, σ) = (0, 20)	$\exp[-(x - 0)^2/(2\times20^2)]$
Gaussian	Moderately Soiled	(c, σ) = (50, 15)	$\exp[-(x - 50)^2/(2\times15^2)]$
Gaussian	Heavily Soiled	(c, σ) = (100, 20)	$\exp[-(x - 100)^2/(2\times20^2)]$

Step 4: Worked calculation of membership degrees (step by step)

Procedure: (i) find which part of the curve the input lies on, (ii) substitute x in that part of the formula, (iii) calculate μ.

Worked input: x = 30 % dirt

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Triangular	Lightly Soiled	falling side ($b \le x \le c$)	$(50 - 30)/(50 - 0) = 20/50 = 0.400$	**0.400**
Triangular	Moderately Soiled	rising side ($a \le x < b$)	$(30 - 25)/(50 - 25) = 5/25 = 0.200$	**0.200**
Triangular	Heavily Soiled	x is outside [50, 100]	$\mu = 0$	**0.000**
Trapezoidal	Lightly Soiled	falling side ($c < x \le d$)	$(50 - 30)/(50 - 20) = 20/30 = 0.667$	**0.667**
Trapezoidal	Moderately Soiled	rising side ($a \le x < b$)	$(30 - 25)/(40 - 25) = 5/15 = 0.333$	**0.333**

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Trapezoidal	Heavily Soiled	x is outside [50, 100]	μ = 0	**0.000**
Gaussian	Lightly Soiled	all x	exp[−(30 − 0)²/(2×20²)] = exp[−900/800] = exp(−1.1250) = 0.325	**0.325**
Gaussian	Moderately Soiled	all x	exp[−(30 − 50)²/(2×15²)] = exp[−400/450] = exp(−0.8889) = 0.411	**0.411**
Gaussian	Heavily Soiled	all x	exp[−(30 − 100)²/(2×20²)] = exp[−4900/800] = exp(−6.1250) = 0.002	**0.002**

Worked input: x = 70 % dirt

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Triangular	Lightly Soiled	x is outside [0, 50]	μ = 0	**0.000**
Triangular	Moderately Soiled	falling side (b ≤ x ≤ c)	(75 − 70)/(75 − 50) = 5/25 = 0.200	**0.200**
Triangular	Heavily Soiled	rising side (a ≤ x < b)	(70 − 50)/(100 − 50) = 20/50 = 0.400	**0.400**
Trapezoidal	Lightly Soiled	x is outside [0, 50]	μ = 0	**0.000**
Trapezoidal	Moderately Soiled	falling side (c < x ≤ d)	(75 − 70)/(75 − 60) = 5/15 = 0.333	**0.333**
Trapezoidal	Heavily Soiled	rising side (a ≤ x < b)	(70 − 50)/(80 − 50) = 20/30 = 0.667	**0.667**
Gaussian	Lightly Soiled	all x	exp[−(70 − 0)²/(2×20²)] = exp[−4900/800] = exp(−6.1250) = 0.002	**0.002**
Gaussian	Moderately Soiled	all x	exp[−(70 − 50)²/(2×15²)] = exp[−400/450] = exp(−0.8889) = 0.411	**0.411**
Gaussian	Heavily Soiled	all x	exp[−(70 − 100)²/(2×20²)] = exp[−900/800] = exp(−1.1250) = 0.325	**0.325**

Step 5: Membership table for the assumed input data (x = 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100)

Triangular membership functions

x (% dirt)	Lightly Soiled	Moderately Soiled	Heavily Soiled
0	1.000	0.000	0.000
10	0.800	0.000	0.000
20	0.600	0.000	0.000
30	0.400	0.200	0.000
40	0.200	0.600	0.000
50	0.000	1.000	0.000
60	0.000	0.600	0.200
70	0.000	0.200	0.400
80	0.000	0.000	0.600
90	0.000	0.000	0.800
100	0.000	0.000	1.000

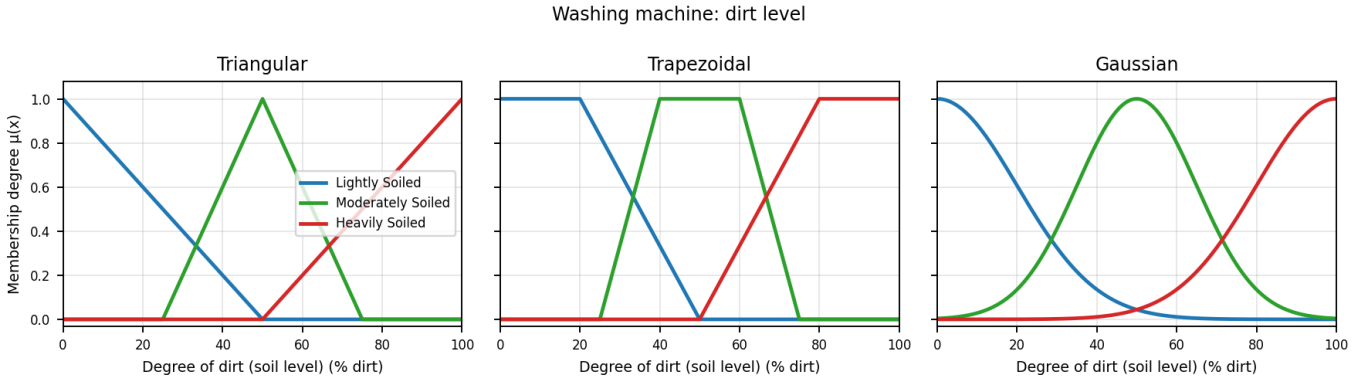
Trapezoidal membership functions

x (% dirt)	Lightly Soiled	Moderately Soiled	Heavily Soiled
0	1.000	0.000	0.000
10	1.000	0.000	0.000
20	1.000	0.000	0.000
30	0.667	0.333	0.000
40	0.333	1.000	0.000
50	0.000	1.000	0.000
60	0.000	1.000	0.333
70	0.000	0.333	0.667
80	0.000	0.000	1.000
90	0.000	0.000	1.000
100	0.000	0.000	1.000

Gaussian membership functions

x (% dirt)	Lightly Soiled	Moderately Soiled	Heavily Soiled
0	1.000	0.004	0.000
10	0.882	0.029	0.000
20	0.607	0.135	0.000
30	0.325	0.411	0.002
40	0.135	0.801	0.011
50	0.044	1.000	0.044
60	0.011	0.801	0.135
70	0.002	0.411	0.325
80	0.000	0.135	0.607
90	0.000	0.029	0.882
100	0.000	0.004	1.000

Step 6: Plot of the membership functions



FINAL ANSWER

Membership functions defined for Lightly Soiled, Moderately Soiled, Heavily Soiled, formulas verified by calculation, and plotted above.

x = 30 % dirt, Triangular: Lightly Soiled = 0.400, Moderately Soiled = 0.200, Heavily Soiled = 0.000

x = 30 % dirt, Trapezoidal: Lightly Soiled = 0.667, Moderately Soiled = 0.333, Heavily Soiled = 0.000

x = 30 % dirt, Gaussian: Lightly Soiled = 0.325, Moderately Soiled = 0.411, Heavily Soiled = 0.002

x = 70 % dirt, Triangular: Lightly Soiled = 0.000, Moderately Soiled = 0.200, Heavily Soiled = 0.400

x = 70 % dirt, Trapezoidal: Lightly Soiled = 0.000, Moderately Soiled = 0.333, Heavily Soiled = 0.667

x = 70 % dirt, Gaussian: Lightly Soiled = 0.002, Moderately Soiled = 0.411, Heavily Soiled = 0.325

Meaning: one crisp input belongs to *two neighbouring sets at the same time* with different degrees (e.g. partly one term and partly the next); this is what a fuzzy system uses in rule evaluation.

Q10. Fuzzy Membership Functions: Room Temperature

Question: *Implement Triangular, Trapezoidal and Gaussian Fuzzy Membership Functions to represent room temperature levels (Cold, Comfortable, Hot) in a fuzzy logic-based room temperature control system. Assume suitable input data and plot the corresponding membership functions.*

Step 1: Choose the universe of discourse and the fuzzy sets

Input variable: **Room temperature**, measured in **°C**. Universe of discourse: $x \in [0, 40]$. Fuzzy sets (linguistic terms): **Cold, Comfortable, Hot**. Each term is represented by three types of membership function (MF).

Step 2: Standard formulas used

MF type	Parameters	Formula
Triangular	a, b, c ($a \le b \le c$)	$\mu(x) = 0$ if $x \le a$ or $x \ge c$; $(x-a)/(b-a)$ if $a \le x \le b$; $(c-x)/(c-b)$ if $b \le x \le c$
Trapezoidal	a, b, c, d ($a \le b \le c \le d$)	$\mu(x) = 0$ if $x \le a$ or $x \ge d$; $(x-a)/(b-a)$ if $a \le x \le b$; 1 if $b \le x \le c$; $(d-x)/(d-c)$ if $c \le x \le d$
Gaussian	c (centre), σ (spread)	$\mu(x) = \exp[-(x-c)^2 / (2\sigma^2)]$

Step 3: Assumed parameters and the resulting formula of every membership function

ASSUMPTION / NOTE

The question says "assume suitable input data", so the ranges and parameters below are **assumed** (they are not given in the question paper or reference PDFs). They are chosen so that neighbouring sets overlap, which is the basic idea of fuzzy logic.

MF type	Fuzzy set	Parameters	Formula with numbers substituted
Triangular	Cold	(a, b, c) = (0, 0, 20)	$(20 - x)/20$ for $0 \le x \le 20$ 0 elsewhere
Triangular	Comfortable	(a, b, c) = (16, 24, 32)	$(x - 16)/8$ for $16 \le x \le 24$ $(32 - x)/8$ for $24 \le x \le 32$ 0 elsewhere
Triangular	Hot	(a, b, c) = (28, 40, 40)	$(x - 28)/12$ for $28 \le x \le 40$ 0 elsewhere
Trapezoidal	Cold	(a, b, c, d) = (0, 0, 12, 20)	1 for $0 \le x \le 12$ $(20 - x)/8$ for $12 \le x \le 20$ 0 elsewhere
Trapezoidal	Comfortable	(a, b, c, d) = (16, 21, 27, 32)	$(x - 16)/5$ for $16 \le x \le 21$ 1 for $21 \le x \le 27$ $(32 - x)/5$ for $27 \le x \le 32$ 0 elsewhere
Trapezoidal	Hot	(a, b, c, d) = (28, 34, 40, 40)	$(x - 28)/6$ for $28 \le x \le 34$ 1 for $34 \le x \le 40$ 0 elsewhere
Gaussian	Cold	(c, σ) = (0, 8)	$\exp[-(x - 0)^2/(2\times8^2)]$
Gaussian	Comfortable	(c, σ) = (24, 4)	$\exp[-(x - 24)^2/(2\times4^2)]$
Gaussian	Hot	(c, σ) = (40, 8)	$\exp[-(x - 40)^2/(2\times8^2)]$

Step 4: Worked calculation of membership degrees (step by step)

Procedure: (i) find which part of the curve the input lies on, (ii) substitute x in that part of the formula, (iii) calculate μ.

Worked input: x = 18 °C

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Triangular	Cold	falling side ($b \le x \le c$)	$(20 - 18)/(20 - 0) = 2/20 = 0.100$	**0.100**
Triangular	Comfortable	rising side ($a \le x < b$)	$(18 - 16)/(24 - 16) = 2/8 = 0.250$	**0.250**
Triangular	Hot	x is outside [28, 40]	$\mu = 0$	**0.000**
Trapezoidal	Cold	falling side ($c < x \le d$)	$(20 - 18)/(20 - 12) = 2/8 = 0.250$	**0.250**
Trapezoidal	Comfortable	rising side ($a \le x < b$)	$(18 - 16)/(21 - 16) = 2/5 = 0.400$	**0.400**
Trapezoidal	Hot	x is outside [28, 40]	$\mu = 0$	**0.000**

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Gaussian	Cold	all x	$\exp[-(18-0)^2/(2\times8^2)] = \exp[-324/128] = \exp(-2.5312) = 0.080$	**0.080**
Gaussian	Comfortable	all x	$\exp[-(18-24)^2/(2\times4^2)] = \exp[-36/32] = \exp(-1.1250) = 0.325$	**0.325**
Gaussian	Hot	all x	$\exp[-(18-40)^2/(2\times8^2)] = \exp[-484/128] = \exp(-3.7812) = 0.023$	**0.023**

Worked input: x = 30 °C

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Triangular	Cold	x is outside [0, 20]	μ = 0	**0.000**
Triangular	Comfortable	falling side (b ≤ x ≤ c)	$(32-30)/(32-24) = 2/8 = 0.250$	**0.250**
Triangular	Hot	rising side (a ≤ x < b)	$(30-28)/(40-28) = 2/12 = 0.167$	**0.167**
Trapezoidal	Cold	x is outside [0, 20]	μ = 0	**0.000**
Trapezoidal	Comfortable	falling side (c < x ≤ d)	$(32-30)/(32-27) = 2/5 = 0.400$	**0.400**
Trapezoidal	Hot	rising side (a ≤ x < b)	$(30-28)/(34-28) = 2/6 = 0.333$	**0.333**
Gaussian	Cold	all x	$\exp[-(30-0)^2/(2\times8^2)] = \exp[-900/128] = \exp(-7.0312) = 0.001$	**0.001**
Gaussian	Comfortable	all x	$\exp[-(30-24)^2/(2\times4^2)] = \exp[-36/32] = \exp(-1.1250) = 0.325$	**0.325**
Gaussian	Hot	all x	$\exp[-(30-40)^2/(2\times8^2)] = \exp[-100/128] = \exp(-0.7812) = 0.458$	**0.458**

Step 5: Membership table for the assumed input data (x = 0, 5, 10, 15, 20, 25, 30, 35, 40)

Triangular membership functions

x (°C)	Cold	Comfortable	Hot
0	1.000	0.000	0.000
5	0.750	0.000	0.000
10	0.500	0.000	0.000
15	0.250	0.000	0.000
20	0.000	0.500	0.000
25	0.000	0.875	0.000
30	0.000	0.250	0.167
35	0.000	0.000	0.583
40	0.000	0.000	1.000

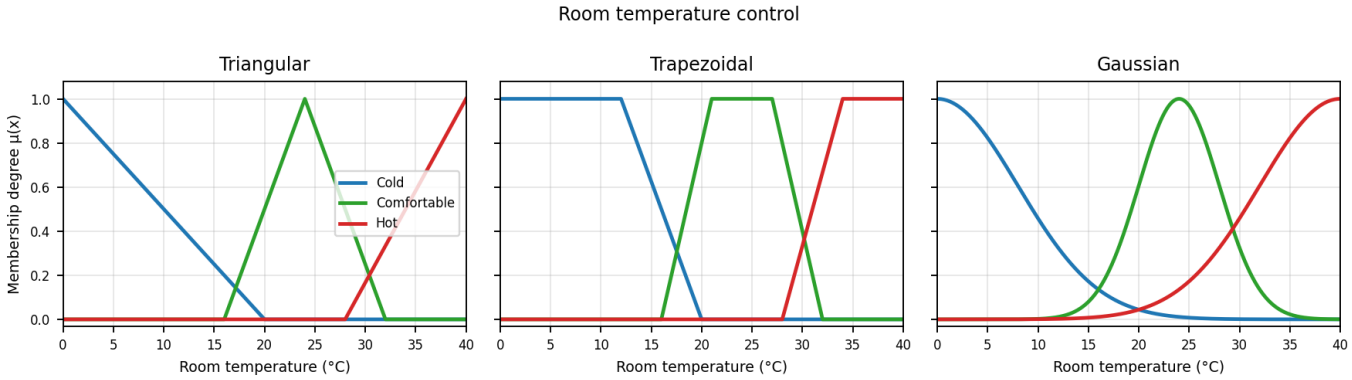
Trapezoidal membership functions

x (°C)	Cold	Comfortable	Hot
0	1.000	0.000	0.000
5	1.000	0.000	0.000
10	1.000	0.000	0.000
15	0.625	0.000	0.000
20	0.000	0.800	0.000
25	0.000	1.000	0.000
30	0.000	0.400	0.333
35	0.000	0.000	1.000
40	0.000	0.000	1.000

Gaussian membership functions

x (°C)	Cold	Comfortable	Hot
0	1.000	0.000	0.000
5	0.823	0.000	0.000
10	0.458	0.002	0.001
15	0.172	0.080	0.008
20	0.044	0.607	0.044
25	0.008	0.969	0.172
30	0.001	0.325	0.458
35	0.000	0.023	0.823
40	0.000	0.000	1.000

Step 6: Plot of the membership functions



FINAL ANSWER

Membership functions defined for Cold, Comfortable, Hot, formulas verified by calculation, and plotted above.

x = 18 °C, Triangular: Cold = 0.100, Comfortable = 0.250, Hot = 0.000

x = 18 °C, Trapezoidal: Cold = 0.250, Comfortable = 0.400, Hot = 0.000

x = 18 °C, Gaussian: Cold = 0.080, Comfortable = 0.325, Hot = 0.023

x = 30 °C, Triangular: Cold = 0.000, Comfortable = 0.250, Hot = 0.167

x = 30 °C, Trapezoidal: Cold = 0.000, Comfortable = 0.400, Hot = 0.333

x = 30 °C, Gaussian: Cold = 0.001, Comfortable = 0.325, Hot = 0.458

Meaning: one crisp input belongs to *two neighbouring sets at the same time* with different degrees (e.g. partly one term and partly the next); this is what a fuzzy system uses in rule evaluation.

Q11. Fuzzy Membership Functions: Traffic Density

Question: *Implement Triangular, Trapezoidal and Gaussian Fuzzy Membership Functions to represent traffic density levels (Low, Medium, High) for a traffic signal control system. Assume suitable input data and plot the corresponding membership functions.*

Step 1: Choose the universe of discourse and the fuzzy sets

Input variable: **Traffic density**, measured in **vehicles/km**. Universe of discourse: $x \in [0, 100]$. Fuzzy sets (linguistic terms): **Low, Medium, High**. Each term is represented by three types of membership function (MF).

Step 2: Standard formulas used

MF type	Parameters	Formula
Triangular	a, b, c ($a \le b \le c$)	$\mu(x) = 0$ if $x \le a$ or $x \ge c$; $(x-a)/(b-a)$ if $a \le x \le b$; $(c-x)/(c-b)$ if $b \le x \le c$
Trapezoidal	a, b, c, d ($a \le b \le c \le d$)	$\mu(x) = 0$ if $x \le a$ or $x \ge d$; $(x-a)/(b-a)$ if $a \le x \le b$; 1 if $b \le x \le c$; $(d-x)/(d-c)$ if $c \le x \le d$
Gaussian	c (centre), σ (spread)	$\mu(x) = \exp[-(x-c)^2 / (2\sigma^2)]$

Step 3: Assumed parameters and the resulting formula of every membership function

ASSUMPTION / NOTE

The question says "assume suitable input data", so the ranges and parameters below are **assumed** (they are not given in the question paper or reference PDFs). They are chosen so that neighbouring sets overlap, which is the basic idea of fuzzy logic.

MF type	Fuzzy set	Parameters	Formula with numbers substituted
Triangular	Low	(a, b, c) = (0, 0, 40)	$(40 - x)/40$ for $0 \le x \le 40$ 0 elsewhere
Triangular	Medium	(a, b, c) = (20, 50, 80)	$(x - 20)/30$ for $20 \le x \le 50$ $(80 - x)/30$ for $50 \le x \le 80$ 0 elsewhere
Triangular	High	(a, b, c) = (60, 100, 100)	$(x - 60)/40$ for $60 \le x \le 100$ 0 elsewhere
Trapezoidal	Low	(a, b, c, d) = (0, 0, 15, 40)	1 for $0 \le x \le 15$ $(40 - x)/25$ for $15 \le x \le 40$ 0 elsewhere
Trapezoidal	Medium	(a, b, c, d) = (20, 40, 60, 80)	$(x - 20)/20$ for $20 \le x \le 40$ 1 for $40 \le x \le 60$ $(80 - x)/20$ for $60 \le x \le 80$ 0 elsewhere
Trapezoidal	High	(a, b, c, d) = (60, 85, 100, 100)	$(x - 60)/25$ for $60 \le x \le 85$ 1 for $85 \le x \le 100$ 0 elsewhere
Gaussian	Low	(c, σ) = (0, 20)	$\exp[-(x - 0)^2/(2\times20^2)]$
Gaussian	Medium	(c, σ) = (50, 15)	$\exp[-(x - 50)^2/(2\times15^2)]$
Gaussian	High	(c, σ) = (100, 20)	$\exp[-(x - 100)^2/(2\times20^2)]$

Step 4: Worked calculation of membership degrees (step by step)

Procedure: (i) find which part of the curve the input lies on, (ii) substitute x in that part of the formula, (iii) calculate μ.

Worked input: x = 30 vehicles/km

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Triangular	Low	falling side ($b \le x \le c$)	$(40 - 30)/(40 - 0) = 10/40 = 0.250$	**0.250**
Triangular	Medium	rising side ($a \le x < b$)	$(30 - 20)/(50 - 20) = 10/30 = 0.333$	**0.333**
Triangular	High	x is outside [60, 100]	$\mu = 0$	**0.000**
Trapezoidal	Low	falling side ($c < x \le d$)	$(40 - 30)/(40 - 15) = 10/25 = 0.400$	**0.400**
Trapezoidal	Medium	rising side ($a \le x < b$)	$(30 - 20)/(40 - 20) = 10/20 = 0.500$	**0.500**
Trapezoidal	High	x is outside [60, 100]	$\mu = 0$	**0.000**

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Gaussian	Low	all x	$\exp[-(30-0)^2/(2\times20^2)] = \exp[-900/800] = \exp(-1.1250) = 0.325$	**0.325**
Gaussian	Medium	all x	$\exp[-(30-50)^2/(2\times15^2)] = \exp[-400/450] = \exp(-0.8889) = 0.411$	**0.411**
Gaussian	High	all x	$\exp[-(30-100)^2/(2\times20^2)] = \exp[-4900/800] = \exp(-6.1250) = 0.002$	**0.002**

Worked input: x = 70 vehicles/km

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Triangular	Low	x is outside [0, 40]	μ = 0	**0.000**
Triangular	Medium	falling side (b ≤ x ≤ c)	(80 − 70)/(80 − 50) = 10/30 = 0.333	**0.333**
Triangular	High	rising side (a ≤ x < b)	(70 − 60)/(100 − 60) = 10/40 = 0.250	**0.250**
Trapezoidal	Low	x is outside [0, 40]	μ = 0	**0.000**
Trapezoidal	Medium	falling side (c < x ≤ d)	(80 − 70)/(80 − 60) = 10/20 = 0.500	**0.500**
Trapezoidal	High	rising side (a ≤ x < b)	(70 − 60)/(85 − 60) = 10/25 = 0.400	**0.400**
Gaussian	Low	all x	$\exp[-(70-0)^2/(2\times20^2)] = \exp[-4900/800] = \exp(-6.1250) = 0.002$	**0.002**
Gaussian	Medium	all x	$\exp[-(70-50)^2/(2\times15^2)] = \exp[-400/450] = \exp(-0.8889) = 0.411$	**0.411**
Gaussian	High	all x	$\exp[-(70-100)^2/(2\times20^2)] = \exp[-900/800] = \exp(-1.1250) = 0.325$	**0.325**

Step 5: Membership table for the assumed input data (x = 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100)

Triangular membership functions

x (vehicles/km)	Low	Medium	High
0	1.000	0.000	0.000
10	0.750	0.000	0.000
20	0.500	0.000	0.000
30	0.250	0.333	0.000
40	0.000	0.667	0.000
50	0.000	1.000	0.000
60	0.000	0.667	0.000
70	0.000	0.333	0.250
80	0.000	0.000	0.500
90	0.000	0.000	0.750
100	0.000	0.000	1.000

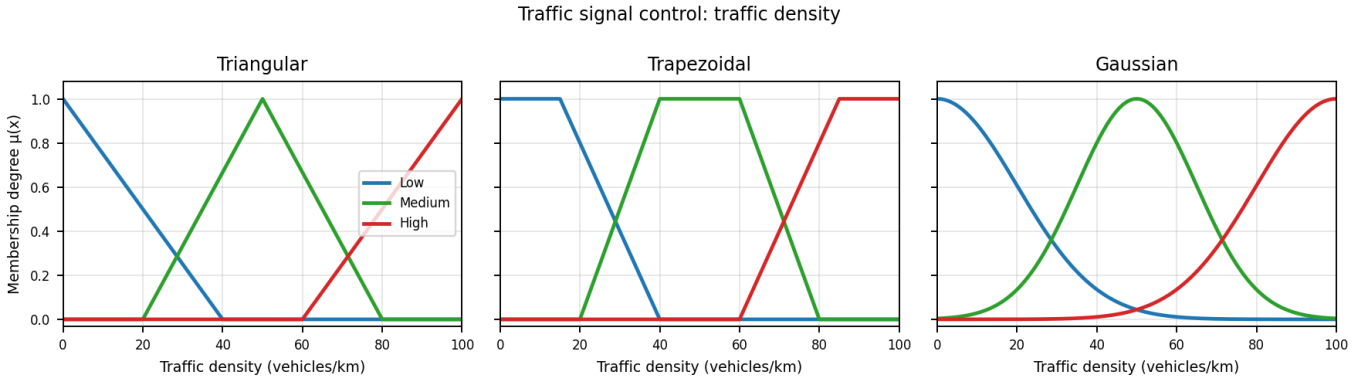
Trapezoidal membership functions

x (vehicles/km)	Low	Medium	High
0	1.000	0.000	0.000
10	1.000	0.000	0.000
20	0.800	0.000	0.000
30	0.400	0.500	0.000
40	0.000	1.000	0.000
50	0.000	1.000	0.000
60	0.000	1.000	0.000
70	0.000	0.500	0.400
80	0.000	0.000	0.800
90	0.000	0.000	1.000
100	0.000	0.000	1.000

Gaussian membership functions

x (vehicles/km)	Low	Medium	High
0	1.000	0.004	0.000
10	0.882	0.029	0.000
20	0.607	0.135	0.000
30	0.325	0.411	0.002
40	0.135	0.801	0.011
50	0.044	1.000	0.044
60	0.011	0.801	0.135
70	0.002	0.411	0.325
80	0.000	0.135	0.607
90	0.000	0.029	0.882
100	0.000	0.004	1.000

Step 6: Plot of the membership functions



FINAL ANSWER

Membership functions defined for Low, Medium, High, formulas verified by calculation, and plotted above.

x = 30 vehicles/km, Triangular: Low = 0.250, Medium = 0.333, High = 0.000

x = 30 vehicles/km, Trapezoidal: Low = 0.400, Medium = 0.500, High = 0.000

x = 30 vehicles/km, Gaussian: Low = 0.325, Medium = 0.411, High = 0.002

x = 70 vehicles/km, Triangular: Low = 0.000, Medium = 0.333, High = 0.250

x = 70 vehicles/km, Trapezoidal: Low = 0.000, Medium = 0.500, High = 0.400

x = 70 vehicles/km, Gaussian: Low = 0.002, Medium = 0.411, High = 0.325

Meaning: one crisp input belongs to *two neighbouring sets at the same time* with different degrees (e.g. partly one term and partly the next); this is what a fuzzy system uses in rule evaluation.

Q12. Fuzzy Membership Functions: Washing Machine (Soil Level)

Question: *Implement Triangular, Trapezoidal and Gaussian Fuzzy Membership Functions to represent washing conditions (Lightly Soiled, Moderately Soiled, Heavily Soiled) in a fuzzy logic-based washing machine system. Assume suitable input data and plot the corresponding membership functions.*

Step 1: Choose the universe of discourse and the fuzzy sets

Input variable: **Degree of dirt (soil level)**, measured in **% dirt**. Universe of discourse: $x \in [0, 100]$. Fuzzy sets (linguistic terms): **Lightly Soiled, Moderately Soiled, Heavily Soiled**. Each term is represented by three types of membership function (MF).

Step 2: Standard formulas used

MF type	Parameters	Formula
Triangular	a, b, c ($a \le b \le c$)	$\mu(x) = 0$ if $x \le a$ or $x \ge c$; $(x-a)/(b-a)$ if $a \le x \le b$; $(c-x)/(c-b)$ if $b \le x \le c$
Trapezoidal	a, b, c, d ($a \le b \le c \le d$)	$\mu(x) = 0$ if $x \le a$ or $x \ge d$; $(x-a)/(b-a)$ if $a \le x \le b$; 1 if $b \le x \le c$; $(d-x)/(d-c)$ if $c \le x \le d$
Gaussian	c (centre), σ (spread)	$\mu(x) = \exp[\ -(x-c)^2 / (2\sigma^2)\ ]$

Step 3: Assumed parameters and the resulting formula of every membership function

ASSUMPTION / NOTE

The question says "assume suitable input data", so the ranges and parameters below are **assumed** (they are not given in the question paper or reference PDFs). They are chosen so that neighbouring sets overlap, which is the basic idea of fuzzy logic.

MF type	Fuzzy set	Parameters	Formula with numbers substituted
Triangular	Lightly Soiled	(a, b, c) = (0, 0, 50)	$(50 - x)/50$ for $0 \le x \le 50$ 0 elsewhere
Triangular	Moderately Soiled	(a, b, c) = (25, 50, 75)	$(x - 25)/25$ for $25 \le x \le 50$ $(75 - x)/25$ for $50 \le x \le 75$ 0 elsewhere
Triangular	Heavily Soiled	(a, b, c) = (50, 100, 100)	$(x - 50)/50$ for $50 \le x \le 100$ 0 elsewhere
Trapezoidal	Lightly Soiled	(a, b, c, d) = (0, 0, 20, 50)	1 for $0 \le x \le 20$ $(50 - x)/30$ for $20 \le x \le 50$ 0 elsewhere
Trapezoidal	Moderately Soiled	(a, b, c, d) = (25, 40, 60, 75)	$(x - 25)/15$ for $25 \le x \le 40$ 1 for $40 \le x \le 60$ $(75 - x)/15$ for $60 \le x \le 75$ 0 elsewhere
Trapezoidal	Heavily Soiled	(a, b, c, d) = (50, 80, 100, 100)	$(x - 50)/30$ for $50 \le x \le 80$ 1 for $80 \le x \le 100$ 0 elsewhere
Gaussian	Lightly Soiled	(c, σ) = (0, 20)	$\exp[-(x - 0)^2/(2\times20^2)]$
Gaussian	Moderately Soiled	(c, σ) = (50, 15)	$\exp[-(x - 50)^2/(2\times15^2)]$
Gaussian	Heavily Soiled	(c, σ) = (100, 20)	$\exp[-(x - 100)^2/(2\times20^2)]$

Step 4: Worked calculation of membership degrees (step by step)

Procedure: (i) find which part of the curve the input lies on, (ii) substitute x in that part of the formula, (iii) calculate μ.

Worked input: x = 30 % dirt

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Triangular	Lightly Soiled	falling side ($b \le x \le c$)	$(50 - 30)/(50 - 0) = 20/50 = 0.400$	**0.400**
Triangular	Moderately Soiled	rising side ($a \le x < b$)	$(30 - 25)/(50 - 25) = 5/25 = 0.200$	**0.200**
Triangular	Heavily Soiled	x is outside [50, 100]	$\mu = 0$	**0.000**
Trapezoidal	Lightly Soiled	falling side ($c < x \le d$)	$(50 - 30)/(50 - 20) = 20/30 = 0.667$	**0.667**
Trapezoidal	Moderately Soiled	rising side ($a \le x < b$)	$(30 - 25)/(40 - 25) = 5/15 = 0.333$	**0.333**

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Trapezoidal	Heavily Soiled	x is outside [50, 100]	μ = 0	**0.000**
Gaussian	Lightly Soiled	all x	exp[−(30 − 0)²/(2×20²)] = exp[−900/800] = exp(−1.1250) = 0.325	**0.325**
Gaussian	Moderately Soiled	all x	exp[−(30 − 50)²/(2×15²)] = exp[−400/450] = exp(−0.8889) = 0.411	**0.411**
Gaussian	Heavily Soiled	all x	exp[−(30 − 100)²/(2×20²)] = exp[−4900/800] = exp(−6.1250) = 0.002	**0.002**

Worked input: x = 70 % dirt

MF type	Fuzzy set	Part of the curve used	Substitution and calculation	μ(x)
Triangular	Lightly Soiled	x is outside [0, 50]	μ = 0	**0.000**
Triangular	Moderately Soiled	falling side (b ≤ x ≤ c)	(75 − 70)/(75 − 50) = 5/25 = 0.200	**0.200**
Triangular	Heavily Soiled	rising side (a ≤ x < b)	(70 − 50)/(100 − 50) = 20/50 = 0.400	**0.400**
Trapezoidal	Lightly Soiled	x is outside [0, 50]	μ = 0	**0.000**
Trapezoidal	Moderately Soiled	falling side (c < x ≤ d)	(75 − 70)/(75 − 60) = 5/15 = 0.333	**0.333**
Trapezoidal	Heavily Soiled	rising side (a ≤ x < b)	(70 − 50)/(80 − 50) = 20/30 = 0.667	**0.667**
Gaussian	Lightly Soiled	all x	exp[−(70 − 0)²/(2×20²)] = exp[−4900/800] = exp(−6.1250) = 0.002	**0.002**
Gaussian	Moderately Soiled	all x	exp[−(70 − 50)²/(2×15²)] = exp[−400/450] = exp(−0.8889) = 0.411	**0.411**
Gaussian	Heavily Soiled	all x	exp[−(70 − 100)²/(2×20²)] = exp[−900/800] = exp(−1.1250) = 0.325	**0.325**

Step 5: Membership table for the assumed input data (x = 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100)

Triangular membership functions

x (% dirt)	Lightly Soiled	Moderately Soiled	Heavily Soiled
0	1.000	0.000	0.000
10	0.800	0.000	0.000
20	0.600	0.000	0.000
30	0.400	0.200	0.000
40	0.200	0.600	0.000
50	0.000	1.000	0.000
60	0.000	0.600	0.200
70	0.000	0.200	0.400
80	0.000	0.000	0.600
90	0.000	0.000	0.800
100	0.000	0.000	1.000

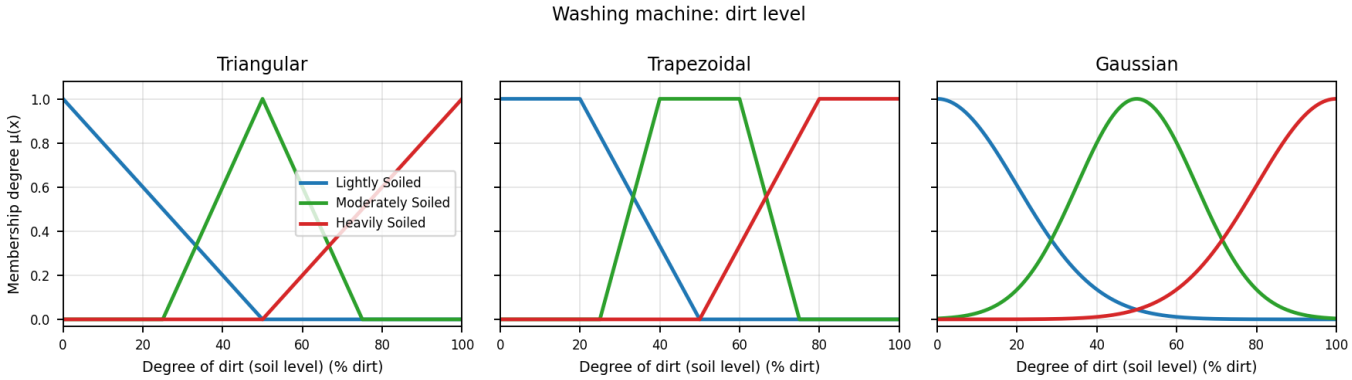
Trapezoidal membership functions

x (% dirt)	Lightly Soiled	Moderately Soiled	Heavily Soiled
0	1.000	0.000	0.000
10	1.000	0.000	0.000
20	1.000	0.000	0.000
30	0.667	0.333	0.000
40	0.333	1.000	0.000
50	0.000	1.000	0.000
60	0.000	1.000	0.333
70	0.000	0.333	0.667
80	0.000	0.000	1.000
90	0.000	0.000	1.000
100	0.000	0.000	1.000

Gaussian membership functions

x (% dirt)	Lightly Soiled	Moderately Soiled	Heavily Soiled
0	1.000	0.004	0.000
10	0.882	0.029	0.000
20	0.607	0.135	0.000
30	0.325	0.411	0.002
40	0.135	0.801	0.011
50	0.044	1.000	0.044
60	0.011	0.801	0.135
70	0.002	0.411	0.325
80	0.000	0.135	0.607
90	0.000	0.029	0.882
100	0.000	0.004	1.000

Step 6: Plot of the membership functions



FINAL ANSWER

Membership functions defined for Lightly Soiled, Moderately Soiled, Heavily Soiled, formulas verified by calculation, and plotted above.

x = 30 % dirt, Triangular: Lightly Soiled = 0.400, Moderately Soiled = 0.200, Heavily Soiled = 0.000

x = 30 % dirt, Trapezoidal: Lightly Soiled = 0.667, Moderately Soiled = 0.333, Heavily Soiled = 0.000

x = 30 % dirt, Gaussian: Lightly Soiled = 0.325, Moderately Soiled = 0.411, Heavily Soiled = 0.002

x = 70 % dirt, Triangular: Lightly Soiled = 0.000, Moderately Soiled = 0.200, Heavily Soiled = 0.400

x = 70 % dirt, Trapezoidal: Lightly Soiled = 0.000, Moderately Soiled = 0.333, Heavily Soiled = 0.667

x = 70 % dirt, Gaussian: Lightly Soiled = 0.002, Moderately Soiled = 0.411, Heavily Soiled = 0.325

Meaning: one crisp input belongs to *two neighbouring sets at the same time* with different degrees (e.g. partly one term and partly the next); this is what a fuzzy system uses in rule evaluation.

Q13. A* Search: A → H

Question: *Implement A* Search to find the path from the start node to the goal node. Heuristic: A(5), B(4), C(3), D(3), E(3), F(2), G(1), H(0). Edge costs: A-B 2, A-C 3, B-D 3, B-E 2, C-F 2, C-G 3, D-H 3, E-H 4, F-H 3, G-H 2.*

ASSUMPTION / NOTE

The question does not name the start and goal nodes. From the data (h(H) = 0 and the similar Q3), **start = A and goal = H** is assumed. Tie-break rule (assumed): if two nodes have equal f, select the one with the smaller h (closer to the goal).

Step 1: Data

Node	A	B	C	D	E	F	G	H
h(n)	5	4	3	3	3	2	1	0

Neighbours with edge cost: A: B(2), C(3) | B: A(2), D(3), E(2) | C: A(3), F(2), G(3) | D: B(3), H(3) | E: B(2), H(4) | F: C(2), H(3) | G: C(3), H(2).

Step 2: Formula

f(n) = g(n) + h(n), where g(n) = actual cost from A to n, h(n) = estimated cost from n to H. OPEN is sorted by smallest f. g of a child = g(parent) + edge cost. If a node is reached again with a smaller g it is updated; with an equal or larger g it is ignored.

Step 3: Execution (each OPEN entry is written as node[g + h = f])

Step	OPEN before selection (sorted)	Select	Expansion: children with g, h, f	CLOSED after
1	A[0+5=5]	A	B: g=0+2=2, h=4, **f=6** C: g=0+3=3, h=3, **f=6**	A
2	C[3+3=6], B[2+4=6] (tie f=6, C has smaller h)	C	F: g=3+2=5, h=2, **f=7** G: g=3+3=6, h=1, **f=7** (A already CLOSED)	A, C
3	B[2+4=6], F[5+2=7], G[6+1=7]	B	D: g=2+3=5, h=3, **f=8** E: g=2+2=4, h=3, **f=7** (A already CLOSED)	A, C, B
4	G[6+1=7], F[5+2=7], E[4+3=7], D[5+3=8] (tie f=7: smallest h first → G)	G	H: g=6+2=8, h=0, **f=8** (C already CLOSED)	A, C, B, G
5	F[5+2=7], E[4+3=7], H[8+0=8], D[5+3=8]	F	H again: g=5+3=8, f=8. Equal to existing g=8 ⇒ no improvement, ignored	A, C, B, G, F
6	E[4+3=7], H[8+0=8], D[5+3=8]	E	H again: g=4+4=8, f=8. Equal ⇒ ignored	A, C, B, G, F, E
7	H[8+0=8], D[5+3=8] (tie f=8, H has h=0)	**H = GOAL**	Goal selected for expansion ⇒ STOP	A, C, B, G, F, E, H

Step 4: Path and cost

Parents: H ← G ← C ← A. Path = **A → C → G → H**. Cost g(H) = 3 + 3 + 2 = **8**.

Step 5: Check that 8 is really the minimum (all paths from A to H)

Path	Cost calculation	Total g
A → B → D → H	2 + 3 + 3	8
A → B → E → H	2 + 2 + 4	8
A → C → F → H	3 + 2 + 3	8
A → C → G → H	3 + 3 + 2	8

FINAL ANSWER

Optimal path (as found by A*): **A → C → G → H** with total cost **8**.

Nodes expanded in order: A, C, B, G, F, E, then H selected.

All four paths from A to H cost 8, so each of them is also optimal (like the "two optimal paths" remark in the reference solution); the one returned depends on the tie-break rule.

Q14. Depth-First Search (DFS): A → G

Question: *Implement DFS to find the path from start node A to goal node G for the given tree-shaped graph.*

Step 1: Read the graph from the figure

Tree edges: A–B, A–C, B–D, B–E, E–H, C–F, C–G. (The hand-written labels "O" and "a" in the figure are read as **D** and **G**.) Children are visited **left to right**.

Node	Neighbours
A	B, C
B	D, E
C	F, G
D	– (leaf)
E	H
F	– (leaf)
G	– (leaf, GOAL)
H	– (leaf)

Step 2: Method

- DFS uses a **STACK (LIFO)**: pop the top node, push its children so that the **left child is on top** (push right child first).
- DFS goes as deep as possible; when a leaf is reached it backtracks automatically by popping the next node.

Step 3: Execution (stack shown top → bottom)

Step	Pop (current)	Goal?	Children pushed	Stack after step	Visited order
0	–	–	A pushed	[A]	–
1	A	No	C then B (B on top)	[B, C]	A
2	B	No	E then D (D on top)	[D, E, C]	A, B
3	D	No	none (leaf) → backtrack	[E, C]	A, B, D
4	E	No	H	[H, C]	A, B, D, E
5	H	No	none (leaf) → backtrack	[C]	A, B, D, E, H
6	C	No	G then F (F on top)	[F, G]	A, B, D, E, H, C
7	F	No	none (leaf) → backtrack	[G]	A, B, D, E, H, C, F
8	G	**YES**	–	[ ]	A, B, D, E, H, C, F, G

Step 4: Path from A to G (only the branch that leads to G, ignoring dead ends B, D, E, H, F)

G ← C ← A ⇒ **A → C → G**.

FINAL ANSWER

DFS traversal order: **A → B → D → E → H → C → F → G**

Path found from A to G: **A → C → G** (2 edges). Nodes visited before reaching goal: 8 (including G).